# Module 1 Fine Tuning LLMs

*Student study notes and worked examples*

**Course CE453 | LLM Production and Deployment | Module duration 8 hours**

These notes explain how to adapt a pretrained language model to a task, reduce the resources needed for inference, and select training settings using evidence. Work through the examples, attempt the learning activities, and then check the answer guidance.

### Syllabus coverage

| Unit | Topic | What you should be able to do |
|---|---|---|
| 1.1 | Introduction to fine tuning | Explain the need for adaptation and plan a task specific training workflow. |
| 1.2 | Optimization for faster inference | Compare compression methods, calculate weight storage and evaluate speed and quality. |
| 1.3 | Hyperparameter tuning | Explain training settings, interpret learning behaviour and select an experiment using validation evidence. |

### Essential vocabulary

| Term | Simple meaning |
|---|---|
| Parameter or weight | A numerical value learned by the model. |
| Token | A unit processed by the tokenizer; it can be a word, part of a word or punctuation. |
| Inference | Using a trained model to produce an output. |
| Loss | A training objective that measures prediction error; lower is usually better for that objective. |
| Hyperparameter | A setting selected by the developer rather than learned as a model weight. |
| Generalization | Performing well on suitable examples that were not used for training. |

Study method: Read the explanation, reproduce each worked calculation without looking, and explain why the answer is sensible. For analysis activities, connect observations to possible causes before suggesting a correction.

**UNIT 1.1**

# Why fine tuning is useful

Fine-tuning continues training from an existing pretrained model using data chosen for a particular task or domain. It changes model parameters so that the model becomes better suited to the desired behaviour.

Analogy: A person who already knows English can learn to write laboratory reports in a particular format. Similarly, an LLM can retain its general language abilities while learning a specialized output style or task.

| Approach | What changes | Illustration |
|---|---|---|
| Pretraining | Model weights are learned through large scale training. | Learn general language patterns from broad text collections. |
| Prompting | The input instructions or examples change; weights stay fixed. | Ask an existing model to return one sentiment label. |
| Fine tuning | Weights or a selected set of adaptation parameters are trained. | Train on many reviewed examples of the required labels or output format. |

### Tasks that benefit from adaptation

- Classification: assign a document to a known category, such as sports, business or science.
- Information extraction: identify fields such as invoice number, date and total amount.
- Summarization: produce concise reports using an agreed style and structure.
- Instruction following: learn to produce outputs that consistently match specified directions.

### Important distinctions

Task-specific fine-tuning targets a particular output task. Domain adaptation targets the language and patterns of a subject area. Supervised fine-tuning uses input-output examples, while instruction fine-tuning uses instructions and desired responses. These descriptions can overlap; they are not mutually exclusive categories.

Fine-tuning can improve relevance, consistency and formatting. It requires representative data and evaluation, and can also reduce performance if data or training choices are poor. It does not guarantee factual correctness or automatically provide newly published information.

Reasoning habit: Before proposing fine-tuning, identify the behaviour that needs to improve and check whether clearer instructions already solve the problem.

UNIT 1.1

# Preparing data for adaptation

Data quality strongly affects what the model learns. A large collection of inconsistent examples can teach the wrong behaviour. Begin with a clear task definition and examples that represent how the application will actually be used.

### A simple preparation workflow

- Define the input and expected output. Decide the valid labels or required output fields.
- Collect representative examples, including ordinary, difficult and ambiguous cases.
- Remove duplicates, repair incorrect labels and standardize the output format.
- Handle missing information consistently. For example, represent an absent invoice date as null rather than guessing it.
- Remove personal or confidential content that is not needed for the learning task.
- Split related examples together so that duplicate documents or the same conversation do not appear in multiple splits.

| Split | Purpose | How it is used |
|---|---|---|
| Training | Learn model parameters. | Used in gradient updates. |
| Validation | Compare settings and checkpoints. | Used during development; no direct gradient updates. |
| Test | Estimate final performance on unseen data. | Kept separate from repeated tuning decisions. |

### Worked dataset split

A collection contains 1,200 independent examples. An illustrative 80 percent training, 10 percent validation and 10 percent test split gives 960 training examples, 120 validation examples and 120 test examples. The appropriate split depends on dataset size and structure.

### Illustrative extraction record

Instruction: Extract the invoice number and total amount.
Input: Invoice INV73. Total payable INR 2450.
Expected output: {"invoice_number": "INV73", "total_inr": 2450}

Data leakage occurs when information from evaluation examples influences training or tuning in a way that makes evaluation unrealistically easy. Split before creating closely related augmented examples, and reserve the final test set for final evaluation.

**UNIT 1.1**

# Choosing a fine tuning method

| Method | Parameters updated | Main resource implication |
|---|---|---|
| Full fine tuning | All model weights. | Usually requires substantial gradient and optimizer memory. |
| Selective fine tuning | A selected subset, such as certain layers. | Resource use depends on which weights remain trainable. |
| PEFT | A comparatively small adaptation parameter set. | Reduces trainable parameters and associated training state. |
| LoRA | Small low rank update matrices in selected modules. | Keeps base weights frozen while learning task adaptations. |
| QLoRA | LoRA adaptation parameters with a frozen quantized base model. | Reduces base weight storage as well as trainable state. |

PEFT means Parameter-Efficient Fine-Tuning. LoRA means Low-Rank Adaptation. It represents an update to a selected weight matrix using smaller trainable matrices. The original base weights remain frozen during this adaptation [1].

QLoRA trains LoRA adapters through a frozen base model stored in 4-bit form. Its original method includes NF4 quantization and other memory-saving techniques. Four-bit weight storage does not imply that every computation, adapter parameter or training state is 4-bit [2].

### Worked trainable parameter percentage

Suppose a base model contains 400 million parameters and an adaptation adds 2 million trainable parameters. The adapter has 0.5 percent as many parameters as the base model:

$$\frac{2 \text{ million}}{400 \text{ million}} \times 100 = 0.5\%$$

If the denominator includes both base and adapter parameters, the exact trainable percentage is 2 / 402 × 100, approximately 0.498 percent. State which denominator you use.

This percentage does not give the total memory saving. Base weights, activations and temporary buffers still consume memory. A small adapter file also requires a compatible base model for inference.

Select a method by comparing task quality, available hardware, training cost and the need to maintain multiple task adaptations. No method is guaranteed to be best for every dataset.

**UNIT 1.1**

# Worked adaptation workflow

Example application: An organization wants to convert invoice text into a small set of structured fields. The existing model sometimes adds explanatory sentences or guesses missing values. The target is accurate extraction in a consistent format.

- Define success: exact field correctness, valid output structure and appropriate handling of missing fields.
- Collect and review examples from varied invoice layouts, vendors and writing styles. Keep related invoice templates together when designing the data split.
- Use consistent instructions and expected outputs. Include missing-field examples and difficult number formats.
- Evaluate the unchanged model first. This baseline shows whether adaptation produces a real improvement.
- Choose full fine-tuning or a resource-appropriate PEFT method. Tokenize examples with the matching tokenizer.
- Run training with recorded settings. For supervised generation, compute the intended response loss using appropriate masking and padding handling.
- Evaluate checkpoints on validation data and inspect field errors. Select the best supported candidate.
- Evaluate the selected candidate once on the reserved test set and compare quality, latency and memory against the baseline.

### How a training step works

The model processes a batch and predicts the desired output tokens. A loss function measures the error. Backpropagation computes gradients for trainable parameters, and the optimizer updates those parameters. This process repeats across batches and epochs.

### Model explanation

Fine-tuning reuses a model that already understands language and adapts it using reviewed invoice examples. Consistent data teaches the required fields and output format. Separate validation data helps compare training choices, while a reserved test set checks generalization. A PEFT method can reduce training resource requirements by updating a small adaptation parameter set. The final decision should consider field correctness and inference performance, not training loss alone.

Expected benefit: more consistent extraction on representative inputs. Limitation: unusual layouts, incorrect OCR and examples outside the training distribution can still cause errors.

**UNIT 1.2**

# Inference performance and compression

Inference is the process of using a trained model. For a generative LLM, it includes processing the input and producing output tokens. An efficient application must meet its quality requirements while using acceptable time and memory.

| Measurement | Meaning | Interpretation |
|---|---|---|
| Latency | Time taken for a request or a defined part of it. | Lower is preferable for the same workload. |
| Time to first token | Delay before the first output token appears. | Important for perceived responsiveness. |
| Throughput | Work completed per unit time, such as tokens per second. | Higher can support more total work. |
| Memory use | Space occupied by weights and runtime state. | Determines whether the workload fits available hardware. |

### Main compression approaches

| Approach | What changes | Key qualification |
|---|---|---|
| Quantization | Numerical representation uses fewer bits. | Quality and speed depend on the method and execution support. |
| Pruning | Selected weights or structures are removed. | Sparsity does not automatically produce faster execution. |
| Distillation | A student model learns from a teacher. | The student needs training and may lose some capabilities. |

### Other influences on inference

Longer inputs increase processing work, while longer outputs require more generation steps. Batching can improve aggregate throughput, but waiting for a batch can increase individual latency. Caching reusable results can avoid repeated work, provided the cached result remains valid.

A KV cache stores attention keys and values during autoregressive generation to avoid recomputing earlier states. It consumes memory and generally grows with active sequences and their lengths.

Training memory additionally includes gradients, optimizer state and saved activations. Therefore, fitting a model for inference does not prove that full fine-tuning will fit on the same device.

Always compare techniques using the same hardware and a clearly defined workload. A smaller file is evidence of reduced storage, not proof of a proportional latency reduction.

**UNIT 1.2**

# Understanding quantization

Quantization maps numerical values to a smaller set of representable values. Reducing bits per weight can reduce model-weight storage. FP32 uses 32 bits per value, FP16 and BF16 use 16 bits, and INT8 uses 8 bits. Four-bit schemes use 4-bit codes, but their numerical formats and metadata differ.

### A toy example of numerical approximation

Consider a simple uniform quantizer with scale 0.1 and zero point 0. A weight of 0.26 is mapped to integer code 3 and reconstructed as 0.30. The absolute approximation error is 0.04. This illustrates rounding error; practical LLM quantizers can use more complex schemes, and NF4 is not this uniform INT4 example.

| Approach | When approximation is introduced | Main consideration |
|---|---|---|
| Post Training Quantization | After a model has been trained. | Often convenient; some methods require representative calibration data. |
| Quantization Aware Training | Quantization effects are simulated during training. | Can help adaptation to low precision but requires an additional training process. |

### Storage precision and compute precision

A model can store base weights in a compressed representation while using another precision for parts of computation. Some layers or values may remain at higher precision. Always distinguish weight storage from arithmetic precision [4].

### Weight memory formula

$$\text{Weight memory in bytes} = \frac{N \times b}{8}$$

N is the number of parameters and b is the number of bits per parameter. Divide bytes by 10 to the power 9 for decimal GB. For binary GiB, divide by 2 to the power 30. Do not mix the units.

$$\text{Reduction percentage} = \frac{\text{Original memory} - \text{New memory}}{\text{Original memory}} \times 100$$

These formulas describe ideal weight storage when all counted parameters use the stated precision. They exclude metadata, unquantized components, activations, KV cache and runtime overhead.

**UNIT 1.2**

# Worked memory and speed calculations

### Example 1 Weight storage reduction

A hypothetical model has 750 million parameters. Compare FP32 weight storage with INT8 weight storage, using decimal GB and excluding all overhead.

$$\text{FP32 memory} = \frac{750{,}000{,}000 \times 32}{8} = 3{,}000{,}000{,}000 \text{ bytes} = 3 \text{ GB}$$

$$\text{INT8 memory} = \frac{750{,}000{,}000 \times 8}{8} = 750{,}000{,}000 \text{ bytes} = 0.75 \text{ GB}$$

$$\text{Memory reduction} = \frac{3 - 0.75}{3} \times 100 = 75\%$$

The saved weight storage is 2.25 GB, and the compression ratio is 3 / 0.75 = 4. The same parameter count remains; the storage representation changes.

### Example 2 A different precision change

For 2.4 billion parameters, FP16 weight storage is 2.4 billion × 2 bytes = 4.8 GB. INT8 storage is 2.4 billion × 1 byte = 2.4 GB. The reduction is 50 percent, and the compression ratio is 2.

### Example 3 Latency improvement

Under the same hardware and workload, an optimization reduces measured request latency from 600 milliseconds to 360 milliseconds.

$$\text{Speedup} = \frac{600}{360} \approx 1.67$$

$$\text{Latency reduction} = \frac{600 - 360}{600} \times 100 = 40\%$$

The result is approximately 1.67 times as fast for this measurement. A 40 percent latency reduction is not the same as a 40 percent increase in processing rate.

### Interpretation

Report storage, latency and quality separately. Changing precision can improve memory use without producing the expected speedup if conversion overhead or execution support limits the benefit. Benchmark the complete intended workload rather than predicting speed from file size.

UNIT 1.2

# Pruning and knowledge distillation

## Pruning

Pruning removes selected weights or structures judged less important for the target behaviour. The resulting model must be evaluated because removed components can affect quality.

| Type | What is removed | Execution implication |
|---|---|---|
| Unstructured pruning | Individual weight values. | Creates sparse matrices; ordinary dense kernels may not run faster. |
| Structured pruning | Groups such as channels, heads or other supported structures. | Can reduce executed work when the resulting model and kernels support it. |

A typical workflow is to evaluate a baseline, choose a pruning method and amount, prune, optionally retrain to recover quality, and benchmark the result. More aggressive pruning can increase quality loss.

## Knowledge distillation

Knowledge distillation trains a student using information supplied by a teacher model. The teacher may provide soft prediction distributions or generated responses. A smaller student can be easier to deploy, although it may not reproduce all teacher capabilities [3].

For a sentiment task, a teacher might assign probabilities of 0.65 to positive, 0.25 to neutral and 0.10 to negative. These soft targets communicate more information than the single hard label positive. In a generative task, teacher responses can instead become reviewed training targets for the student.

- Choose a student and data that represent the intended task.
- Obtain suitable teacher outputs or distributions and check their quality.
- Train the student and evaluate it on separate, representative data.
- Measure whether reduced resource use is worth any loss of task quality.

## Selecting an approach

Quantization is a natural candidate when weight storage is the main constraint. Pruning is useful when supported structural or sparse execution can reduce work. Distillation is useful when a smaller model is acceptable and training data or teacher outputs are available. Combinations are possible, but every combined change requires evaluation.

**UNIT 1.3**

# Hyperparameters and their effects

Parameters are learned during training. Hyperparameters control how training or generation operates. Their effects depend on the model, task and dataset; no single setting is best for every application.

| Training setting | Meaning | Possible effect of changing it |
|---|---|---|
| Learning rate | Scale of optimizer updates. | Too high can destabilize learning; too low can make adaptation slow or ineffective. |
| Epochs | Passes through the training dataset. | Too few may undertrain; too many can overfit. |
| Micro batch size | Examples processed per device before accumulation. | Affects memory use and gradient noise. |
| Gradient accumulation | Micro batches combined before an update. | Increases effective batch size without loading the whole batch at once. |
| Sequence length | Maximum tokens processed per example. | Longer lengths usually increase memory and computation; truncation can remove useful content. |
| Weight decay | A regularization setting in suitable optimizers. | Can discourage overly large weights, but excessive values may harm learning. |
| Dropout | Training time random removal of selected activations. | Can reduce overfitting, while excessive dropout can hinder learning. |
| Warmup | Initial period of gradually increasing learning rate. | Can make early training updates less abrupt. |
| LoRA rank | Capacity of the low rank update. | Higher rank increases adaptation parameters and memory; quality gains are not guaranteed. |

### Generation settings are different

Temperature, top-p, top-k and maximum output tokens affect generation. Temperature changes a sampling distribution; top-p and top-k restrict sampling candidates. They do not replace learning-rate or epoch tuning. Their effect depends on the decoding method, and some are ignored when sampling is disabled.

Reducing sampling randomness can improve consistency in some tasks, but it does not guarantee factual correctness. Maximum output length can limit unnecessary generation, but a limit that is too small can truncate a correct response.

**UNIT 1.3**

# Effective batch size and tuning workflow

### Effective batch size

$$B = m \times a \times d$$

B is effective batch size, m is micro batch size, a is the number of accumulation steps, and d is the number of data-parallel devices.

This formula assumes compatible batches and data-parallel training, with each data-parallel device processing a distinct micro batch. Model sharding across several GPUs does not automatically multiply the data-parallel batch size.

Example: A micro batch of 3, accumulation over 4 micro batches and 2 data-parallel devices gives an effective batch of $3 \times 4 \times 2 = 24$ examples. Accumulation saves activation memory compared with processing the full effective batch at once, but is not identical to a large batch in every implementation.

### Worked training step count

For one device, 900 training examples and micro batch size 15 give 60 micro batches per epoch. With accumulation over 3 micro batches, there are 20 optimizer updates per epoch. Four epochs give 80 updates. This example assumes complete batches and exact division; partial batches and framework rules can change the count.

### A disciplined tuning process

- Fix a representative validation set and a task metric before comparing trials.
- Record a baseline model and baseline training configuration.
- Start with a small experiment budget and vary a few relevant settings.
- Compare validation quality, resource use and training stability.
- For close results, repeat useful trials with different random seeds to assess variability.
- Select the supported candidate and reserve the final test set for the final check.

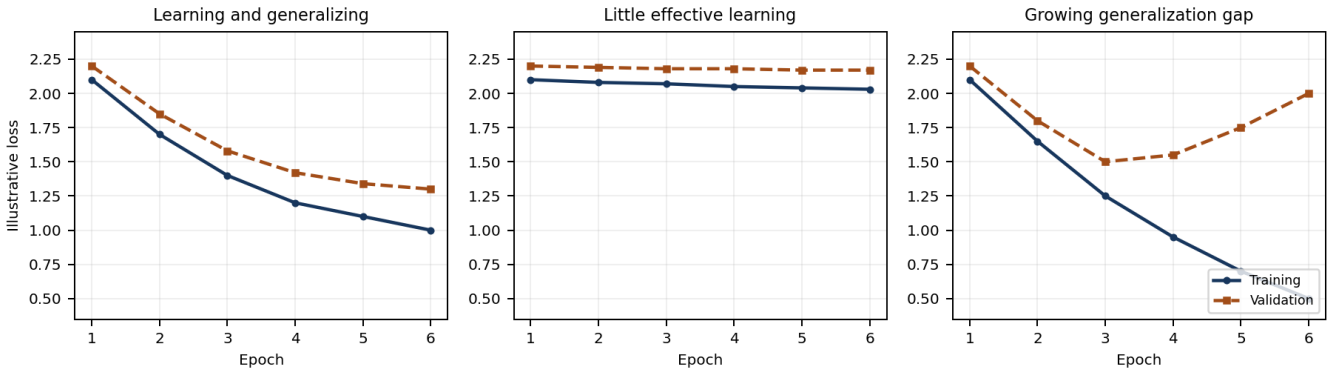
| Search method | How candidates are chosen | Main trade off |
|---|---|---|
| Manual search | Developer chooses each trial. | Simple, but can miss useful combinations. |
| Grid search | Try every listed combination. | Systematic, but trial counts grow quickly. |
| Random search | Sample combinations from defined choices. | Can explore more settings within a fixed budget. |

For example, 3 learning rates × 2 batch sizes × 2 epoch settings produce 12 grid candidates. Keep evaluation conditions consistent and log all runs, including unsuccessful ones.

**UNIT 1.3**

# Reading learning curves

Training loss measures the objective on training examples. Validation loss measures the same objective on separate examples. Compare trends only when tokenization, masking and evaluation procedures are compatible.




Illustrative curves for explanation only. These values are not experimental results.

| Observation | Interpretation to investigate | Possible response |
|---|---|---|
| Both losses fall | Useful learning and improving generalization. | Continue monitoring; gains may eventually level off. |
| Both stay high and nearly flat | Undertraining, unsuitable settings, data problems or an implementation error. | Check labels, masking, gradients and trainable parameters before changing settings. |
| Training improves while validation worsens | Possible overfitting; also inspect evaluation noise and data mismatch. | Use early stopping, improve data and compare regularization or training duration. |
| Loss jumps or becomes invalid | Possible unstable updates, numerical issues or malformed batches. | Inspect logs and data; review learning rate and precision settings. |

Early stopping ends training after a chosen period without meaningful validation improvement. Keep the best validation checkpoint rather than assuming the final checkpoint is best.

Diagnosis requires evidence. A batch-size change alone does not prove that overfitting has been solved, and one unusual validation point should not automatically determine a decision.

# Evaluating quality and performance

A useful model must solve the task on unseen examples and meet operational needs. Training loss alone cannot demonstrate either requirement.

| Task or concern | Useful evidence |
|---|---|
| Classification | Accuracy and, where relevant, precision, recall and F1 for individual classes. |
| Structured extraction | Correct fields, valid schema and appropriate handling of missing values. |
| Text generation | Correctness, relevance, completeness and instruction following using a clear rubric. |
| Efficiency | Weight storage, peak runtime memory, latency and throughput on the target system. |

### A fair comparison

Use the same evaluation examples, model input format and scoring rules. For performance tests, document hardware, software, input lengths, output lengths, batch size, concurrency and measurement boundaries. Warm up the service and collect enough observations to characterize normal variation.

### Worked comparison using illustrative measurements

| Measurement | Baseline | Compressed candidate |
|---|---|---|
| Task accuracy | 94.0% | 93.5% |
| Measured request latency | 450 ms | 300 ms |
| Model artifact size | 1.60 GB | 0.45 GB |

The accuracy change is a decrease of 0.5 percentage points. Latency speedup is 450 / 300 = 1.5. Artifact size falls by (1.60 − 0.45) / 1.60 × 100 = 71.875 percent. These are separate findings, not evidence of a universal trade-off.

A deployment decision still needs explicit application requirements and runtime memory measurements. Artifact size does not establish total GPU memory use, and the table does not establish performance at every concurrency level.

### What to record for every useful experiment

Record the base model and tokenizer, dataset version and split, training method, settings, random seed, selected checkpoint, validation results, hardware, runtime and final evaluation results. This record makes comparisons reproducible and helps explain unexpected changes.

# Independent learning activities

Attempt these activities before reading the answer guidance. Use complete explanations and show calculations. The activities develop explanation, application, comparison and diagnosis skills using fresh examples.

### Activity 1 Explaining adaptation

Write a short explanation of how a pretrained model can learn to label product reviews as positive, neutral or negative. Include the role of examples, dataset separation, a suitable adaptation method and final evaluation.

### Activity 2 Comparing training approaches

A laboratory wants to build several document-labeling applications using one base model and limited training memory. Compare full fine-tuning with LoRA and explain what must still be stored and evaluated when adapters are used.

### Activity 3 Calculating model weight storage

Estimate ideal weight storage for 2.5 billion parameters in FP32 and FP16. Find the storage saved and percentage reduction. Use decimal GB and exclude overhead. Explain why these figures do not give total inference memory.

### Activity 4 Interpreting an inference improvement

A measured optimization reduces latency from 900 ms to 600 ms under the same test conditions. Calculate speedup and latency reduction, then describe the additional quality and resource measurements needed to interpret the result.

### Activity 5 Counting batches and updates

On one device, use 640 training examples, micro batch size 2, gradient accumulation over 8 micro batches and 3 epochs. Calculate effective batch size, micro batches per epoch, optimizer updates per epoch and total updates. Assume exact division.

### Activity 6 Preparing an evaluation split

Divide 600 independent examples into 70 percent training, 15 percent validation and 15 percent test. Explain the purpose of each split and how repeated documents could invalidate the evaluation.

### Activity 7 Diagnosing weak learning

Training and validation losses remain almost unchanged throughout a short run. Develop a diagnosis plan that checks both the implementation and training settings before deciding whether to extend training.

### Activity 8 Comparing experimental evidence

Three comparable trials report validation losses of 0.94, 0.87 and 0.92. Explain which trial is the initial candidate for further evaluation and why validation loss alone is insufficient for a deployment decision.

# Answer guidance for activities 1 to 4

### Activity 1 Model explanation

Fine-tuning adapts a model that already understands language to a specific labeling task. Collect representative product reviews with consistent positive, neutral and negative labels. Correct errors and separate training, validation and test examples without placing duplicate reviews across splits. Train using full fine-tuning or a suitable PEFT method and monitor validation performance. For generated labels, teach and check the exact allowed label format. Compare the selected checkpoint with the original model on the reserved test set using accuracy and class-wise measures. Inspect mixed or ambiguous reviews rather than relying only on an overall score. Also measure inference cost and latency. The intended improvement is more reliable labeling on new reviews, not memorization of training examples.

### Activity 2 Comparison and recommendation

Full fine-tuning updates all base weights and usually requires more trainable state. LoRA freezes the base weights and trains smaller low rank updates in selected modules. Separate adapters can support several tasks while sharing a compatible base model, reducing adaptation storage relative to storing a full copy per task. LoRA is therefore a useful candidate for the stated memory constraint, but task quality must be checked. The base model, matching tokenizer, adapter configuration and relevant software are still needed. Activations and runtime buffers still consume memory; adapters alone do not eliminate the base model cost. Some inference workflows merge an adapter into supported base weights, but compatibility and resulting memory must be verified.

### Activity 3 Worked calculation

FP32 uses 4 bytes per parameter, so storage is 2.5 billion × 4 = 10 billion bytes = 10 GB. FP16 uses 2 bytes, so storage is 5 GB. Saved weight storage is 5 GB. Percentage reduction is (10 − 5) / 10 × 100 = 50 percent. Actual inference memory additionally includes activations, KV cache, buffers and runtime overhead.

### Activity 4 Worked calculation and interpretation

Speedup is 900 / 600 = 1.5. Latency reduction is (900 − 600) / 900 × 100, approximately 33.33 percent. The request takes two-thirds of its original time. Compare task quality before and after optimization, peak memory, throughput and performance at realistic concurrency. Document input and output lengths and verify that the improvement was not obtained simply by generating shorter or incomplete answers.

# Answer guidance for activities 5 to 8

### Activity 5 Worked calculation

| Quantity | Calculation | Answer |
|---|---|---|
| Effective batch size | 2 × 8 × 1 | 16 examples |
| Micro batches per epoch | 640 / 2 | 320 |
| Optimizer updates per epoch | 320 / 8 | 40 |
| Total optimizer updates | 40 × 3 | 120 |

The model processes eight micro batches before each optimizer update. These counts assume one device, exact division and no skipped updates. Three epochs process 1,920 training-example presentations in total.

### Activity 6 Split and explanation

Training: 600 × 0.70 = 420 examples. Validation: 600 × 0.15 = 90 examples. Test: 90 examples. Training updates parameters, validation guides settings and checkpoint selection, and the test set supports the final independent evaluation. Duplicates or related versions of the same document across splits can make unseen-data performance appear better than it really is. Group related records before splitting.

### Activity 7 Diagnosis plan

Nearly flat losses indicate little effective learning but do not identify a single cause. First verify that inputs and labels are correct, the intended output tokens contribute to the loss, gradients are computed and the intended parameters are trainable. Confirm that optimizer updates actually occur and that loss values are logged correctly. Then examine whether the learning rate is too small, training is too short or the adaptation capacity is unsuitable. A very small training subset can be used as a diagnostic check for whether the training process can learn at all; this is not final quality evidence. Change one suspected cause or use controlled trials, then compare validation behaviour. Extend training only after checking these points.

### Activity 8 Evidence based selection

The trial with validation loss 0.87 is the initial candidate because it has the lowest loss under the stated comparable conditions. This does not establish the highest task accuracy or the best deployment performance. Check the task metric, representative outputs, stability across runs, latency and memory. If the loss differences are small, assess evaluation variability. Select using validation evidence and predefined goals, then use the reserved test set for the final evaluation rather than repeatedly tuning against it.

# Revision guide and further reading

### Core ideas to remember

- Fine-tuning adapts model behaviour using relevant examples; evaluation establishes whether it helped.
- Full fine-tuning updates base weights. LoRA trains low rank updates while the base remains frozen. QLoRA combines such adaptation with a quantized frozen base.
- Quantization changes representation, pruning removes selected components, and distillation trains a student from a teacher.
- Learning rate controls update scale. Epochs control repeated exposure. Batch settings affect memory and optimization.
- Validation evidence guides development. A separate test set supports the final check.

### Frequent mistakes and corrections

| Mistake | Correction |
|---|---|
| Lower training loss always means a better model. | Check validation behaviour and task performance on unseen examples. |
| An adapter is a complete standalone model. | It normally needs its compatible base model and configuration. |
| A smaller model file guarantees proportional speedup. | Measure execution on the intended hardware and workload. |
| Weight memory is the total GPU memory requirement. | Include runtime state, buffers, activations and KV cache. |
| A temperature change fine tunes the model. | Temperature changes sampling behaviour, not learned weights. |

### Writing a complete technical answer

Define the concept, explain the mechanism, apply it to the stated example and justify the result. Show numerical units and assumptions. For a comparison, use common criteria. For diagnosis, connect each observation to a possible cause and a test that could confirm it.

### Further reading

[1] Hu et al. LoRA Low Rank Adaptation of Large Language Models, 2021. https://arxiv.org/abs/2106.09685

[2] Dettmers et al. QLoRA Efficient Finetuning of Quantized LLMs, 2023. https://arxiv.org/abs/2305.14314

[3] Hinton, Vinyals and Dean. Distilling the Knowledge in a Neural Network, 2015. https://arxiv.org/abs/1503.02531

[4] Hugging Face Transformers documentation. Bitsandbytes quantization. https://huggingface.co/docs/transformers/quantization/bitsandbytes